

基于智能体后训练的 NL2SQL 技术研究

一、技术背景

在大数据时代，海量的结构化、半结构化数据被存储在关系型数据库中，通过结构化查询语句（Structured Query Language, SQL）可以完成对数据库的各种操作。然而，SQL 的学习曲线对于初学者来说是陡峭的，这给非专业人士使用数据库带来困难。在这样的背景下诞生了自然语言转结构化查询语言技术（Natural Language to SQL, NL2SQL），旨在降低数据库技术的上手门槛^[1]。具体来说，给定自然语言问题 Q 、数据库模式 $\mathcal{S}$ ，NL2SQL 的目标是预测 Q 对应的 SQL 查询 y 。NL2SQL 的评测指标一般使用执行准确率，即比对 y 与 参考 SQL 在数据库中的执行结果，计算完全一致的比例^[2]。

作为自然语言处理（Natural Language Processing, NLP）与数据库技术的交集，NL2SQL 伴随 NLP 的发展而不断进步。最早的 NL2SQL 依赖于人工设计的规则和模板匹配，在计算效率上具有优势，但是只适合一些简单场景^[3]；在 Transformer 问世后，一些工作将 NL2SQL 建模为 Seq2Seq 的机器翻译任务，将 NL2SQL 的性能提高到了新的水平^[4]；现代的 NL2SQL 则普遍采用仅解码器结构的大语言模型（Large Language Models, LLM），在预测 SQL 的准确性和泛化性上都取得了长足的进步。依据是否存在 LLM 的参数更新，这些现代方法可以分为两类：1）无需后训练，而是专注于 Agent 架构设计、精巧的提示词工程以及 Harness 工程，通过 API 调用 Claude Opus 等旗舰 LLM 来完成 NL2SQL 任务^[5]；2）基于开源 LLM 做后训练，涉及监督微调（Supervised Fine-tuning, SFT）^[6]、带有可验证奖励的强化学习（Reinforcement Learning with Verifiable Rewards, RLVR）^[7]、同策略蒸馏（On Policy Distillation, OPD）^[8]等后训练技术。由于第一类方法依赖于商业旗舰模型，涉及大量的 Token 消耗，并且存在数据安全问题，所以不适合企业场景；而第二类方法不仅成本低，还可以实现私有化的服务部署，因此更具有研究价值。

然而，当前基于 LLM 后训练的方法一般只是简单地将 NL2SQL 建模为“预测 SQL 中的每一个 token”任务： $y = \pi_{\theta}(Q, \mathcal{S})$ ，然后通过标准的 SFT、RLVR 等后训练流程提高 LLM 预测 SQL 的执行准确性。整个 NL2SQL 过程是一次性、无交互的，即 LLM 只是根据自然语言问题 Q 和数据库模式 $\mathcal{S}$ ，来自回归地预测 SQL 文本本身，无法利用数据库引擎去执行 SQL、获取执行结果，因此预测 SQL 的语法正确性、语义正确性都无法得到保证。为了实现具备 SQL 执行和改错功能的 NL2SQL 智能体，最直观的做法是基于 ReAct 架构，将 LLM 纳入代码编排的工作流，在 SQL 执行出错时反馈错误信息，并提示 LLM 进行改错，直到预测 SQL 执行无误或达到指定迭代轮数终止^[9]。这类方法的代表工作是 ReFoRCE^[10]。然而，由于当前的后训练类 NL2SQL 方法只训练了 LLM 预测 SQL 的能力，因此改错功能完全取决于 LLM 本身的泛化能力，这在企业真实的应用场景下缺乏可靠性。作为对比，我们的研究旨在应用 SFT、RLVR、OPD 等 LLM 后训练技术，直接强化开源 LLM 的 SQL 编写、SQL 改错等 NL2SQL 智能体能力，并在智能体 RL 的奖励设计、信用分配等方面做出创新，实现专门适配 NL2SQL 技术的智能体后训练方案，得到具备强大自主探索和决策能力的 NL2SQL 智能体，在预测性能、运行成本和数据安全等方面都具有优势。

二、技术方案

本项目的核心目标是构建一个具备自主思考、工具调用与多轮纠错能力的 NL2SQL 智能体。为了使模型具备自主环境感知、错误反馈推理及复杂 SQL 迭代修正的能力，本项目的技术路线整体架构如图 1 所示，包含以下三个阶梯式推进的功能模块。

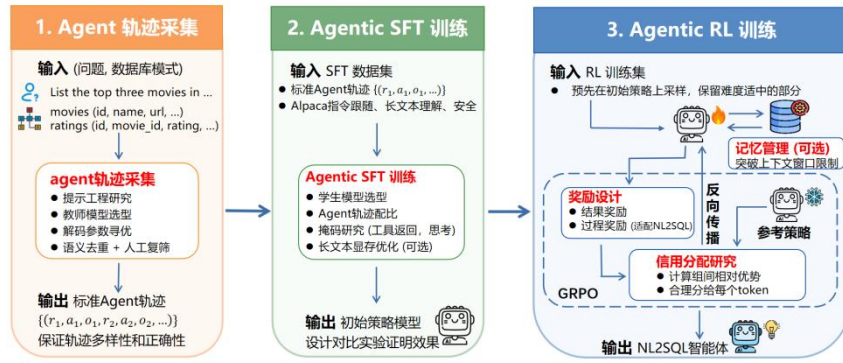


图 1 项目技术路线架构图

(1) Agent 轨迹采集

本模块聚焦于构建 SFT 数据集。以“自然语言问题 + 数据库模式”为输入，通过精细化的提示工程、教师模型选型与解码参数寻优，在真实 SQLite 环境中采集多轮交互轨迹^[11]。通过语义去重与人工复筛，最终构建出高质量、多样化、包含“思考-行动-观察”的标准 NL2SQL Agent 轨迹数据集。

(2) Agentic SFT 训练

本模块使用模块一采集的轨迹数据集，基于领先的开源编码模型（如 Qwen2.5-Coder-14B-Instruct）进行全量微调^[12]。通过合理的轨迹配比与损失掩码技术，强制模型激活 Agent 交互规范，并建立初步的反思纠错逻辑，其产出的权重将作为模块三强化学习的初始策略。

(3) Agentic RL 训练

本模块是智能体核心能力突破的关键。利用大语言模型后训练框架，在 SFT 模型的基础上引入基于群体相对优势策略优化（GRPO）的强化学习算法^[13]。通过设计精确的结果奖励与中间过程奖励，并结合面向 Token 级别的信用分配研究，引导智能体在没有专家轨迹指导的未知边界内，通过自主探索和环境反馈激发智能体的推理与纠错能力，最终形成高鲁棒性的 NL2SQL 智能体^[14]。

1. Agent 轨迹采集

本模块聚焦于 SFT 数据集的构建，旨在打造一个高质量、多样化、可复现的 Agent 执行轨迹数据集，为后续的 Agentic SFT 训练阶段提供核心的数据支撑。整个技术流程以“自然语言问题 + 数据库模式”作为初始输入，最终输出结构化的标准化轨迹序列。在具体实施中，首先通过提示工程研究约束模型行为以确保格式规范，随后进行教师模型选型筛选出推理与 SQL 生成能力最强的底层基座，并结合解码参数寻优在轨迹的正确性与多样性之间找到最佳平衡点；采集到的原始轨迹将进一步通过语义去重与人工复筛过滤冗余、低质样本，从而最终提炼出高价值的标准轨迹集合。

(1) 提示工程研究

提示工程研究的核心目的是让大模型稳定遵循“先推理、再动作、再观察”的结构化行为模式，从而提高输出一致性并降低格式错误导致的无效轨迹^[15]。设计提示模板，对比不同指令形式的效果，并通过强制输出固定结构文本来约束模型行为，最终在样本测试中选出格式正确率和任务成功率最高的模板作为后续实验的稳定基线。

(2) 教师模型选型

教师模型选型的目标是从候选大模型中选出能稳定生成高质量轨迹的底层模型，为后续轨迹采集提供可靠的生成能力。首先根据公开榜单筛选推理能力较强的候选模型，然后在小规模测试集上对比各模型的实际表现，重点评估其 SQL 生成准确性和输出轨迹的规范性，

同时结合成本与其他因素进行权衡，最终选择综合性价比最高的模型用于大规模轨迹采集。

(3) 解码参数寻优

解码参数寻优的目标是在轨迹的正确性与多样性之间找到最佳平衡点，避免模型要么过于保守（总是输出相同路径），要么过于随机（频繁出错）。通过选取温度、top-p、top-k 等关键参数，进而在小规模测试集上进行网格搜索或简单随机采样，同时记录每条轨迹的执行成功率和输出格式的规范性，来筛选出表现均衡的参数组合^[16]。

(4) 语义去重 + 人工复筛

语义去重加人工复筛的目标是去除冗余和低质量的轨迹，保留具有真实多样性的高价值样本，为后续微调或评估提供干净可靠的数据集。先用嵌入模型将每条轨迹转换为向量，再通过聚类或余弦相似度阈值自动过滤掉语义相近的同构轨迹^[17]；然后由人工对剩余轨迹进行复核，重点检查动作是否合理、中间推理是否自洽，剔除存在语法错误或隐含幻觉的样本，并根据任务复杂度分析难度等级，最终形成高质量、多样化的标准轨迹集合。

2. Agentic SFT 训练

本模块聚焦于大模型的行为对齐与初步纠错能力的激活。整个技术流程以第一阶段输出的 SFT 数据集作为初始输入，通过全量微调全面激活 Agent 交互规范并实现格式对齐，最终为模块三 RL 阶段提供初始策略^{[6][18]}。在具体实施中，首先通过学生模型选型明确基座模型，随后在数据配比与掩码策略上进行精细化设计，以在强化特定能力的同时兼顾模型的泛化性；同时，针对超长交互序列带来的计算压力，引入长文本显存优化机制以保障梯度训练的平稳高效；最后，将训练好的模型接入真实环境进行评估，从而最终确定进入强化学习阶段的初始策略。

(1) 学生模型选型

学生模型选型的核心目的是确立进行全量微调的底层基座。方案重点在开源模型中进行多维度横向测评与适配性探索。选用 Qwen2.5-Coder、Qwen3-Coder 等领先的开源编码模型作为基座，分析不同参数量级以及不同模型的训练表现，权衡计算开销和效率，选择模型。旨在通过全量微调的方式，全面激活模型的 Agent 交互规范并实现格式对齐，为后续的特定制能力强化打下坚实的基座基础。

(2) agent 轨迹配比

Agent 轨迹配比的目标是在强化特定能力的同时兼顾模型的泛化性，有效防止通用能力退化。在具体配比上，采用专家轨迹 + 通用数据（Alpaca 类指令跟随、长文本理解）的多组混合数据，设计不同比例的两种轨迹进行训练实验获取最优比例^[19]；在轨迹的选择上，重点分析包含“生成-报错-修正”过程的轨迹，从而提高模型的反思能力与泛化能力。

(3) 损失掩码

引入损失掩码（Loss Masking）技术的核心目的是精细化控制微调框架中的监督信号传递，防止大模型在长序列交互训练中“无脑背诵”非自身生成的外部环境数据。本方案对损失计算的覆盖粒度与边界进行了对比与敏感度研究。在 verl 训练框架^[22]的具体实施中，通过探索遮盖环境返回结果和错误思考过程等方式，使模型在训练时聚焦于自身动作序列与正确思维链的生成，从而显著提高 Agent 面对未知数据库环境时的策略泛化能力与纠错自洽性。

(4) 长文本显存优化（可选）

长文本显存优化机制的目标是应对超长交互序列带来的计算压力，保障长序列梯度训练的平稳高效。面对此问题可启用序列打包（Sequence Packing）与上下文并行（Context Parallelism）技术，在工程实现层面解决全量微调过程中的显存爆炸风险，从而提升高频交互下的训练稳定性。

本模块训练执行完毕后在 BIRD 验证集上评估 Pass@1 和 Pass@10 两个指标，得出模型

在稳定情况下的表现和较高温度下模型的能力上限。测试合格后的模型权重作为下一阶段 AgenticRL 训练的初始权重。

3. Agentic RL 训练

在完成 Agentic SFT 训练后，本模块进一步通过强化学习提升模型在复杂 NL2SQL 场景下的自我验证与纠错能力^[20]。传统 NL2SQL 通常将自然语言问题到 SQL 的生成视为一次性推理过程，而本方案将数据库执行引擎作为可交互环境，使模型能够在生成 SQL 后调用执行工具，根据语法报错、空结果或执行结果反馈进行修正，最终输出可执行的 SQL^[21]。

(1) 强化学习框架与数据集

在算法与框架选型方面，本模块采用工业级分布式后训练框架 verl，算法核心基于 GRPO（群体相对优势策略优化）^[13]。该算法通过让模型对同一个问题生成一组不同的 SQL 轨迹，并通过组内得分对比获得相对优势来更新模型，以此提升高质量轨迹的生成概率。

在训练集样本筛选上，方案根据自然语言问题的难易程度进行过滤。系统预先在模块二得到的初始策略模型上进行轨迹采样，直接过滤掉模型总是能写对的极简样本、总是写不对的极难样本，重点保留难度适中、需要多轮交互的样本入库。这样可以避免模型在无用任务上浪费算力，确保强化学习训练能够高效聚焦于“探索与反思”的核心区间^[22]。

(2) 奖励设计

在 Agentic RL 训练过程中，模型针对同一问题生成多条 Agent 轨迹，并依据最终 SQL 的执行结果计算奖励。本模块的奖励设计以结果奖励为主，包括 SQL 是否可执行、执行结果是否正确、是否出现空结果或非法查询等；同时可适当加入过程奖励，引导模型学习合理调用工具、利用执行反馈修正 SQL^[23]。

(3) 信用分配设计

在多轮交互中，一条轨迹往往包含大量的 Token，而环境给出的结果奖励通常都在最末尾^[24]。为了解决这种稀疏奖励导致的模型训练迷茫问题，本模块设计了 Token 级别的信用分配机制。算法利用 GRPO 的群体相对优势特性，先在同组采样的多条轨迹之间计算出组间相对优势。随后，将最终的环境奖励沿着交互路径一步步向前反向传导，从而把信用精准、合理地分给中间步骤里的每一个关键 Token。这种设计避免了把奖励平均分摊给所有 Token 的弊端，能够有效强化模型在关键决策点上的反思与动态纠错能力^[25]。

三、预期目标

实现一套完整的 NL2SQL Agentic 后训练框架，涵盖轨迹采集、Agentic SFT、Agentic RL 等关键环节，重点探索 Agentic RL 阶段的奖励函数以及信用分配机制。在小米汽车业务数据集上取得 85% 的执行准确率，在 NL2SQL 学术榜单 BIRD 上取得 70% 以上的执行准确率。

四、进度安排

阶段	时间 安排	主要任务	阶段产出
原型系统开发- Agent 轨迹采集	2 周	1.搭建 NL2SQL Agent 运行环境，设计初版 NL2SQL 提示词，实现数据集到 Agent prompt 的统一映射格式 2.接入 SQL 执行器，支持模型生成 SQL 后获取执行反馈 3.实现 NL2SQL 评估和 Agent 轨迹采集功能，过滤错误轨迹、空执行轨迹和过长低质量轨迹	NL2SQL Agent 运行环境、NL2SQL 评估脚本、Agent 轨迹采集脚本、标准 Agent 轨迹数据集

原型系统开发- Agentic SFT 训练	2 周	1.基于轨迹数据集，对基座模型做监督微调 2.对 SQL 执行结果等 observation 内容进行 loss mask，避免模型直接背诵环境反馈； 3.设置不添加 loss mask 的对照组（常规 SFT），在验证集上在线评测两个 SFT 模型的性能,得到初始策略模型	初版 Agentic SFT 模型、 训练配置、 pass@1/pass@k 评测结果
原型系统开发- Agentic RL 训练	4 周	1.基于 verl / RL-Factory 开发 RL pipeline，完成工具调用、环境交互、奖励计算、采样缓存、策略更新等模块，奖励设计采用 a) 只用执行正确性结果奖励 b) 结果奖励+简单的过程奖励两组 2.评测 RL 前后策略模型的 NL2SQL 性能	可运行的 Agentic RL 训练 代码、初版奖励函数版、评 测报告
系统迭代优化- SQL 生成提示 词工程研究	4 周	从 Agent 轨迹采集阶段着手，深入研究提示词工程对 NL2SQL 能力的影响，以验证集上的执行准确率作为标准，不断迭代优化	最终版本 NL2SQL 提示词
系统迭代优化- SFT 优化研究	8 周	1.研究 Agentic SFT 掩码策略对 NL2SQL 模型性能的影响，对于思考内容和 SQL 执行结果，设计不同的掩码策略，比较对性能的影响 2.研究 Agentic SFT 阶段数据配比,通过正向轨迹、SFT 纠错轨迹、边界样本、通用类数据的合理搭配，最大化 SFT 模型的性能 3.研究针对长文本的 SFT 显存优化策略，提升模型在多轮 NL2SQL 交互场景的能力	SFT 掩码方案设计、数据 配比策略
系统迭代优化- RL 优化研究	12 周	1.围绕 SQL 执行准确率、表召回、语法正确性、空结果惩罚和自我修正质量设计奖励 2.探索步骤级或片段级信用分配,缓解长轨迹中优势分配不准确的问题 3.对比不同奖励组合对 pass@1 和 pass@10 的影响，针对性地进行迭代优化	RL 奖励函数设计和信用 分配研究报告、消融实验 报告

参考文献

- [1] ZHONG V, XIONG C, SOCHER R. Seq2SQL: Generating structured queries from natural language using reinforcement learning[C]//Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing. Copenhagen: Association for Computational Linguistics, 2017: 1985-1995.
- [2] YU T, ZHANG R, YASUNAGA M, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task[C]//Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Brussels: Association for Computational Linguistics, 2018: 3911-3921.
- [3] YU T, ZHANG R, YANG K, et al. SyntaxSQLNet: Syntax tree networks for complex and cross-domain text-to-SQL task[C]//Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Brussels: Association for Computational Linguistics, 2018: 1653-1663.
- [4] WANG C, TATSUOKA K, LIU X, et al. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers[C]//Proceedings of the 58th Annual Meeting of the

Association for Computational Linguistics. Online: Association for Computational Linguistics, 2020: 7567-7582.

- [5] KIM J, HWANG H. Harness Engineering: A governance framework for AI-driven software engineering[EB/OL]. SSRN Electronic Journal, 2026[2026-05-26]. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6372119.
- [6] OUYANG L, WU J, JIANG X, et al. Training language models to follow instructions with human feedback[C]//Advances in Neural Information Processing Systems 35. New Orleans: Curran Associates Inc., 2022: 27730-27744.
- [7] WEN X, LIU Z, ZHENG S, et al. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs[J/OL]. arXiv:2506.14245, 2025.
- [8] RUSU A A, COLMENAREJO S G, GULCEHRE C, et al. Policy distillation[J/OL]. arXiv:1511.06295, 2015.
- [9] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing reasoning and acting in language models[C]//International Conference on Learning Representations. Kigali: ICLR, 2023.
- [10] DAHAN E, SHRAGAI B, SHPILMAN T, et al. ReFoRCE: A text-to-SQL agent with self-refinement and error correction[J/OL]. arXiv:2402.01070, 2024.
- [11] QIN Y, LI Y, LIANG S, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs[J/OL]. arXiv:2307.16789, 2023.
- [12] YANG A, YANG B, ZHANG B, et al. Qwen2.5 technical report[J/OL]. arXiv:2412.15115, 2024.
- [13] SHAO Z, WANG P, ZHU Q, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models[J/OL]. arXiv:2402.03300, 2024.
- [14] SCHICK T, DESSÌ R, ATTANASIO G, et al. Toolformer: Language models can teach themselves to use tools[J/OL]. arXiv:2302.04761, 2023.
- [15] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems 35. New Orleans: Curran Associates Inc., 2022: 24824-24837.
- [16] MEISTER C, PIMENTEL T, WIHER G, et al. Locally typical sampling[J]. Transactions of the Association for Computational Linguistics, 2023, 11: 102-121.
- [17] MUENNIGHOFF N, TAZI N, MAGNE L, et al. MTEB: Massive text embedding benchmark[C]//Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. Dubrovnik: Association for Computational Linguistics, 2023: 2014-2037.
- [18] CHUNG H W, HOU L, LONGPRE S, et al. Scaling instruction-finetuned language models[J]. Journal of Machine Learning Research, 2024, 25(70): 1-53.
- [19] TAORI R, GULRAJANI I, ZHANG T, et al. Stanford Alpaca: An instruction-following LLaMA model[EB/OL]. (2023-03-13)[2026-05-26].

<https://crfm.stanford.edu/2023/03/13/alpaca.html>.

- [20] LI J, HUI B, QU G, et al. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs[C]//Advances in Neural Information Processing Systems 36. New Orleans: Curran Associates Inc., 2023: 42330-42357.
- [21] YANG J, MACCALLUM A, CHEN M, et al. InterCode: Standardizing and benchmarking interactive coding with execution feedback[C]//Advances in Neural Information Processing Systems 36. New Orleans: Curran Associates Inc., 2023: 73784-73797.
- [22] LE H, WANG Y, GOTMARE A D, et al. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning[C]//Advances in Neural Information Processing Systems 35. New Orleans: Curran Associates Inc., 2022: 21314-21328.
- [23] SHAO Z, YU P, WANG M, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning[J/OL]. arXiv:2501.12948, 2025.
- [24] ZHENG L, CHI Y, DONG Y, et al. Process reward models for large language model reasoning[J/OL]. arXiv:2501.07301, 2025.
- [25] WANG Z, XIE T, MIN S, et al. Improving long-horizon reasoning via credit assignment in language model reinforcement learning[J/OL]. arXiv:2502.01234, 2025.